DAA: LAB 2

Stack and Queue

Name: Sameer Gupta
Roll No: 252010036
Date: February 21, 2026

1 Stack and Queue:

```
#include <bits/stdc++.h>
using namespace std;
class Stack{
private:
    int* arr;
    int capacity;
    int sizeStack;
public:
    Stack(){
        capacity=100;
        arr=new int[capacity];
        sizeStack=0;
    }
    ~Stack() {
        delete[] arr;
    }
    void printStack(){
        for(int i=0;i<sizeStack;i++){
            cout<<arr[i]<<" ";
        }
        cout<<endl;
    }
    void push(int num){
        if(sizeStack>=capacity){
            cout<<"Stack overflow"<<endl;
            return;
        }
        arr[sizeStack++]=num;
    }
    void insertAtKthPosition(int num, int k)
    {
        if (k < 0 || k > sizeStack)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        if (sizeStack >= capacity)
        {
            cout << "Overflow: Stack is full!" << endl;
            return;
        }
        for (int i = sizeStack - 1; i >= k; i--)
        {
            arr[i + 1] = arr[i];
```

```
        }

        arr[k] = num;
        sizeStack++;
    }
    bool isPresent(int num){
        for(int i=0;i<sizeStack;i++){
            if(arr[i]==num)return true;
        }
        return false;
    }
    void pop(){
        if(sizeStack<=0){
            cout<<"Stack underflow"<<endl;
            return;
        }
        sizeStack--;
    }
    void deleteFromKthPosition(int k)
    {
        if (k < 0 || k >= sizeStack)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        for (int i = k; i < sizeStack - 1; i++)
        {
            arr[i] = arr[i + 1];
        }
        sizeStack--;
    }
    void updateKthElement(int num, int k)
    {
        if (k < 0 || k >= sizeStack)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        arr[k] = num;
    }
    int peek(){
        if(sizeStack<=0){
            cout<<"Stack is empty"<<endl;
            return -1;
        }
        return arr[sizeStack-1];
    }
    int getLength(){
        return sizeStack;
    }
    bool isEmpty(){
        return sizeStack==0;
    }
};
```

```
class Queue
{
private:
    int *arr;
    int capacity;
    int start;
    int end;
    int sizeQueue;

public:
    Queue()
    {
        capacity = 100;
        arr = new int[capacity];
        start = -1;
        end = -1;
        sizeQueue = 0;
    }
    ~Queue()
    {
        delete[] arr;
    }
    void printQueue()
    {
        if (sizeQueue <= 0)
        {
            cout << "Queue is empty" << endl;
            return;
        }
        cout << "Queue: ";
        for (int i = 0; i < sizeQueue; i++)
        {
            cout << arr[(i + start) % capacity] << " ";
        }
        cout << endl;
    }
    void push(int num)
    {
        if (sizeQueue >= capacity)
        {
            cout << "Queue overflow" << endl;
            return;
        }
        if (sizeQueue == 0)
        {
            start = 0;
            end = 0;
        }
        else
        {
            end = (end + 1) % capacity;
        }
        arr[end] = num;
        sizeQueue++;
```

```
    }
    void insertAtKthPosition(int num, int k)
    {
        if (k < 0 || k > sizeQueue)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        if (sizeQueue >= capacity)
        {
            cout << "Overflow: Queue is full!" << endl;
            return;
        }
        if (sizeQueue == 0) {
            push(num);
            return;
        }
        int idxToInsert = (start + k) % capacity;
        int temp = end;
        for (int i = sizeQueue - 1; i >= k; i--)
        {
            int next = (temp + 1) % capacity;
            arr[next] = arr[temp];

            temp--;
            if (temp == -1) temp = capacity - 1;
        }

        arr[idxToInsert] = num;
        end = (end + 1) % capacity;
        sizeQueue++;
    }
    bool isPresent(int num)
    {
        for (int i = 0; i < sizeQueue; i++)
        {
            if (arr[(i + start) % capacity] == num)
                return true;
        }
        return false;
    }
    void pop()
    {
        if (sizeQueue <= 0)
        {
            cout << "Queue underflow" << endl;
            return;
        }

        start = (start + 1) % capacity;
        sizeQueue--;
        if (sizeQueue == 0){
            start = -1;
            end = -1;
```

```
            sizeQueue = 0;
        }
    }
    void deleteFromKthPosition(int k)
    {
        if (k < 0 || k >= sizeQueue)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        int idxToDelete = (start + k) % capacity;
        int temp = idxToDelete;
        for (int i = k; i < sizeQueue - 1; i++)
        {
            int next = (temp + 1) % capacity;
            arr[temp] = arr[next];
            temp = next;
        }
        sizeQueue--;
        end--;
        if (end == -1) end = capacity - 1;

        if(sizeQueue == 0){
            start = -1;
            end = -1;
        }
    }
    void updateKthElement(int num, int k)
    {
        if (k < 0 || k >= sizeQueue)
        {
            cout << "Invalid Position!" << endl;
            return;
        }
        arr[(start + k) % capacity] = num;
    }
    int peek()
    {
        if (sizeQueue <= 0)
        {
            cout << "Queue is empty" << endl;
            return -1;
        }
        return arr[start];
    }
    int getLength()
    {
        return sizeQueue;
    }
    bool isEmpty()
    {
        return sizeQueue == 0;
    }
};
```

```
int main()
{
   cout << endl;
    cout << "------------------------------------------------------------------" << endl;
    cout << "------------------------------STACK------------------------------" << endl;
    cout << "------------------------------------------------------------------" << endl;
    cout << endl;

    Stack st;
    cout << "--- 1. Initial Push ---" << endl;
    st.push(5);
    st.push(2);
    st.push(3);
    st.push(4);
    st.printStack();

    cout << "\n--- 2. Insert 10 at position 2 ---" << endl;
    st.insertAtKthPosition(10, 2);
    st.printStack();

    cout << "\n--- 3. Delete element at position 0 ---" << endl;
    st.deleteFromKthPosition(0);
    st.printStack();

    cout << "\n--- 4. Update element at position 1 to 99 ---" << endl;
    st.updateKthElement(99, 1);
    st.printStack();

    cout << "\n--- 5. Popping 1 element ---" << endl;
    st.pop();
    st.printStack();

    cout << "\n--- 6. Peek at the top element ---" << endl;
    cout << "Top element is: " << st.peek() << endl;

    cout << "\n--- 7. Get the current length ---" << endl;
    cout << "Stack length is: " << st.getLength() << endl;

    cout << "\n--- 8. Check if the Stack is empty ---" << endl;
    cout << "Is stack empty? " << (st.isEmpty() ? "Yes" : "No") << endl;

    cout << "\n--- 9. Check if elements exist (isPresent) ---" << endl;
    cout << "Is 99 present in stack? " << (st.isPresent(99) ? "Yes" : "No") << endl;
    cout << "Is 50 present in stack? " << (st.isPresent(50) ? "Yes" : "No") << endl;

    cout << endl;
```

```
    cout << "
        ----------------------------------------------------------------"
        << endl;
    cout << "------------------------------QUEUE
        ------------------------------" << endl;
    cout << "
        ----------------------------------------------------------------"
        << endl;
    cout << endl;

    Queue q;
    cout << "--- 1. Initial Push ---" << endl;
    q.push(10);
    q.push(2);
    q.push(8);
    q.push(1);
    q.push(3);
    q.push(5);
    q.push(12);
    q.push(4);
    q.push(14);
    q.printQueue();

    cout << "\n--- 2. Popping 4 elements ---" << endl;
    q.pop();
    q.pop();
    q.pop();
    q.pop();
    q.printQueue();

    cout << "\n--- 3. Pushing more elements ---" << endl;
    q.push(12);
    q.push(4);
    q.push(14);
    q.printQueue();

    cout << "\n--- 4. Insert 67 at position 5 ---" << endl;
    q.insertAtKthPosition(67, 5);
    q.printQueue();

    cout << "\n--- 5. Delete element at position 6 ---" << endl;
    q.deleteFromKthPosition(6);
    q.printQueue();

    cout << "\n--- 6. Update element at position 2 to 100 ---" << endl;
    q.updateKthElement(100, 2);
    q.printQueue();

    cout << "\n--- 7. Peek at the front element ---" << endl;
    cout << "Front element is: " << q.peek() << endl;

    cout << "\n--- 8. Get the current length ---" << endl;
    cout << "Queue length is: " << q.getLength() << endl;

```

```
    cout << "\n--- 9. Check if the Queue is empty ---" << endl;
    cout << "Is queue empty? " << (q.isEmpty() ? "Yes" : "No") << endl;

    cout << "\n--- 10. Check if elements exist (isPresent) ---" << endl;
    cout << "Is 100 present in queue? " << (q.isPresent(100) ? "Yes" : "No") << endl;
    cout << "Is 999 present in queue? " << (q.isPresent(999) ? "Yes" : "No") << endl;

    return 0;
}
```

```
-----------------------------------------------------------------
------------------------------STACK------------------------------
-----------------------------------------------------------------

--- 1. Initial Push ---
5 2 3 4

--- 2. Insert 10 at position 2 ---
5 2 10 3 4

--- 3. Delete element at position 0 ---
2 10 3 4

--- 4. Update element at position 1 to 99 ---
2 99 3 4

--- 5. Popping 1 element ---
2 99 3

--- 6. Peek at the top element ---
Top element is: 3

--- 7. Get the current length ---
Stack length is: 3

--- 8. Check if the Stack is empty ---
Is stack empty? No

--- 9. Check if elements exist (isPresent) ---
Is 99 present in stack? Yes
Is 50 present in stack? No
```

Figure 1: Stack output

```
-----------------------------------------------------------------
-----------------------------QUEUE-----------------------------
-----------------------------------------------------------------

--- 1. Initial Push ---
Queue: 10 2 8 1 3 5 12 4 14

--- 2. Popping 4 elements ---
Queue: 3 5 12 4 14

--- 3. Pushing more elements ---
Queue: 3 5 12 4 14 12 4 14

--- 4. Insert 67 at position 5 ---
Queue: 3 5 12 4 14 67 12 4 14

--- 5. Delete element at position 6 ---
Queue: 3 5 12 4 14 67 4 14

--- 6. Update element at position 2 to 100 ---
Queue: 3 5 100 4 14 67 4 14

--- 7. Peek at the front element ---
Front element is: 3

--- 8. Get the current length ---
Queue length is: 8

--- 9. Check if the Queue is empty ---
Is queue empty? No

--- 10. Check if elements exist (isPresent) ---
Is 100 present in queue? Yes
Is 999 present in queue? No
```

Figure 2: Queue output

DAA Lab2

Name: Sameer Gupta
Roll No: 252010036

www.jeevandeep.in
PAGE NO.
DATE: / /

Aim:
To study and implement operations on linear data structures - stack and queue.

Stack:
A linear structure that follows the LIFO (Last in first Out) principle. Operations occur only at one end (top).

Queue:
A Linear data structure following the FIFO (First In First Out) principle. Elements are added at the rear and removed from the front.

Op	Stack	Queue
Insert	O(1)	O(1)
Delete	O(1)	O(1)
Search	O(n)	O(n)

Conclusion:
Different data structures serve different purposes
Stack: For recursion and undo operations
Queue: For scheduling & buffering

Figure 3: Theory